

高级数据结构第一次作业

学号：202533354 姓名：ZHOU MINGYUAN

1. 稀疏矩阵与转置

(1) 用 Python 嵌套列表表示如下：

```
M = [
    [0, 0, 8, 0],
    [3, 0, 0, 0],
    [0, 0, 1, 0],
    [0, 2, 0, 4],
    [7, 0, 0, 0],
    [0, 0, 0, 0],
    [0, 3, 0, 0],
]
```

行\列	0	1	2	3
0	0	0	8	0
1	3	0	0	0
2	0	0	1	0
3	0	2	0	4
4	7	0	0	0
5	0	0	0	0
6	0	3	0	0

(2) 转置算法：创建一个列数 $\times$ 行数的新矩阵，只在原矩阵元素非 0 时写入对应位置。这样可避免对 0 做额外赋值，结果仍是二维数组逻辑结构。

```
def transpose_sparse_matrix(M):
    rows, cols = len(M), len(M[0])
    T = [[0 for _ in range(rows)] for _ in range(cols)]
    for i in range(rows):
        for j in range(cols):
            if M[i][j] != 0:
                T[j][i] = M[i][j]
    return T
```

转置结果为：

行\列	0	1	2	3	4	5	6
0	0	3	0	0	7	0	0
1	0	0	0	2	0	0	3
2	8	0	1	0	0	0	0
3	0	0	0	4	0	0	0

复杂度：若矩阵为 $m \times n$ ，时间复杂度为 $O(mn)$ ，额外空间复杂度为 $O(mn)$ 。若只保存非零三元组，可进一步做到 $O(n + t)$ ，其中 t 为非零元素个数。

2. 稀疏多项式运算

采用字典表示多项式，key 为指数，value 为系数。该结构能直接按指数合并同类项。

```
A = {1000: 2, 3: 4}
B = {4: 1, 3: 10, 2: -3, 1: 1, 0: 1}
```

A - B 的计算结果:

$A(x)-B(x)=2x^{1000}-x^4-6x^3+3x^2-x-1$

A * B 的计算结果:

$A(x)B(x)=2x^{1004}+20x^{1003}-6x^{1002}+2x^{1001}+2x^{1000}+4x^7+40x^6-12x^5+4x^4+4x^3$

```
def subtract(P, Q):
    R = dict(P)
    for exp, coeff in Q.items():
        R[exp] = R.get(exp, 0) - coeff
        if R[exp] == 0:
            del R[exp]
    return R

def multiply(P, Q):
    R = {}
    for e1, c1 in P.items():
        for e2, c2 in Q.items():
            exp = e1 + e2
            R[exp] = R.get(exp, 0) + c1 * c2
            if R[exp] == 0:
                del R[exp]
    return R
```

复杂度分析：设 A 有 m 项，B 有 n 项。减法只需扫描两个字典，时间复杂度 $O(m+n)$ ，空间复杂度 $O(m+n)$ 。乘法需要两层循环枚举项对，时间复杂度 $O(mn)$ ，结果最多有 mn 个不同指数项，空间复杂度 $O(mn)$ 。

3. 失败函数与 KMP 算法

失败函数也称部分匹配表或前缀函数。对模式串 P 的每个位置 i，记录 $P[0..i]$ 的最长相等真前缀和真后缀长度。匹配失败时，KMP 不回退主串指针，而是根据失败函数移动模式串，从而复用已经比较过的信息。

KMP 的核心过程：先用 $O(m)$ 时间计算模式串长度 m 的失败函数；再扫描文本串 T，若字符匹配就同时前进，若失配就把模式串位置 j 调整为 $fail[j-1]$ 。主串长度为 n 时，总时间复杂度为 $O(n+m)$ ，空间复杂度为 $O(m)$ 。

```
def prefix_function(P):
    fail = [0] * len(P)
    j = 0
    for i in range(1, len(P)):
        while j > 0 and P[i] != P[j]:
            j = fail[j - 1]
        if P[i] == P[j]:
            j += 1
        fail[i] = j
    return fail
```

4. Stack 与 Queue

Stack 是后进先出结构。常见操作是 push、pop、top/peek。Python 中可用 list 实现：append 入栈，pop 出栈，末尾操作均摊 $O(1)$ 。

Queue 是先进先出结构。常见操作是 enqueue、dequeue、front。Python 中更适合用 collections.deque 实现：append 入队，popleft 出队，都是 $O(1)$ 。如果用 list.pop(0)，每次出队都要移动后续元素，时间复杂度为 $O(n)$ 。

项目	Stack	Queue
访问规则	后进先出 LIFO	先进先出 FIFO
插入位置	栈顶	队尾
删除位置	栈顶	队头
典型应用	函数调用、括号匹配、表达式求值	任务调度、BFS、缓冲区
Python 实现	list.append / list.pop	collections.deque.append / popleft

共同点：二者都是线性表的受限形式，只允许在特定端点进行插入或删除。不同点：Stack 强调最近加入的元素先处理，Queue 强调最早加入的元素先处理。

5. 循环队列

循环队列把固定数组首尾相接。front 和 rear 到达数组末尾后通过取模回到开头，避免普通顺序队列因前端空位无法复用而产生假溢出。优点是空间利用率高，入队和出队均为 $O(1)$ 。

```
class CircularQueue:
    def __init__(self, capacity):
        self.data = [None] * capacity
        self.front = 0
        self.rear = 0
        self.size = 0

    def enqueue(self, x):
        if self.size == len(self.data):
            raise OverflowError('queue is full')
        self.data[self.rear] = x
        self.rear = (self.rear + 1) % len(self.data)
        self.size += 1

    def dequeue(self):
        if self.size == 0:
            raise IndexError('queue is empty')
        x = self.data[self.front]
        self.data[self.front] = None
        self.front = (self.front + 1) % len(self.data)
        self.size -= 1
        return x
```

6. 中缀表达式转后缀表达式

表达式： $8*(7-3)/4-(2+3)*(7-4)$ 。优先级：*, /, % 高于 +, -, 同级运算符按左结合处理。

读入	动作	运算符栈	后缀输出
8	输出操作数		8
*	入栈	*	8
(	入栈	*, (	8
7	输出操作数	*, (	8 7
-	入栈	*, (, -	8 7
3	输出操作数	*, (, -	8 7 3
)	弹出到左括号	*	8 7 3 -
/	弹出 *, 再入栈 /	/	8 7 3 - *
4	输出操作数	/	8 7 3 - * 4
-	弹出 /, 再入栈 -	-	8 7 3 - * 4 /
(	入栈	-, (	8 7 3 - * 4 /
2	输出操作数	-, (	8 7 3 - * 4 / 2
+	入栈	-, (, +	8 7 3 - * 4 / 2
3	输出操作数	-, (, +	8 7 3 - * 4 / 2 3
)	弹出到左括号	-	8 7 3 - * 4 / 2 3 +
%	优先级高于 -, 入栈	-, %	8 7 3 - * 4 / 2 3 +
(	入栈	-, %, (	8 7 3 - * 4 / 2 3 +
7	输出操作数	-, %, (	8 7 3 - * 4 / 2 3 + 7
-	入栈	-, %, (, -	8 7 3 - * 4 / 2 3 + 7
4	输出操作数	-, %, (, -	8 7 3 - * 4 / 2 3 + 7 4
)	弹出到左括号	-, %	8 7 3 - * 4 / 2 3 + 7 4 -
结束	依次弹出 %, -		8 7 3 - * 4 / 2 3 + 7 4 - % -

因此后缀表达式为：8 7 3 - * 4 / 2 3 + 7 4 - % -

```
def infix_to_postfix(tokens):
    prec = {'+': 1, '-': 1, '*': 2, '/': 2, '%': 2}
    stack, out = [], []
    for tok in tokens:
        if tok.isdigit():
            out.append(tok)
        elif tok == '(':
            stack.append(tok)
        elif tok == ')':
            while stack and stack[-1] != '(':
                out.append(stack.pop())
            stack.pop()
        else:
            while stack and stack[-1] != '(' and prec[stack[-1]] >= prec[tok]:
                out.append(stack.pop())
            stack.append(tok)
    while stack:
        out.append(stack.pop())
    return out

expr = ['8', '*', '(', '7', '-', '3', ')', '/', '4', '-', '(',
        '2', '+', '3', ')', '%', '(', '7', '-', '4', ')']
print(' '.join(infix_to_postfix(expr)))
```